

WILFRID LAURIER UNIVERSITY
Department of Physics and Computer Science

Implementation of the Fischer et al. (2021) Reinforcement Learning Methodology

CP493 Directed Research Project — Implementation Summary

Prepared by
Ranjot Sandhu

Submitted to
Professor Sukhjit Sehra

May 6, 2026
Waterloo, Ontario, Canada

1. Summary

Fischer et al. (2021), *Sci. Rep.* 11:14445, trained a reinforcement learning agent on a biomechanical model of the human upper extremity and showed that the resulting policy reproduced two emergent regularities of natural human motor control: Fitts' Law ($R\text{-squared} = 0.9986$) and the two-thirds Power Law ($R = 0.84$). Their methodology has six components: SAC as the training algorithm, an adaptive curriculum on goal tolerance, a motor-babbling pre-training phase, the two validation harnesses, and Hill-type muscle dynamics on a 7-DOF arm.

All six components have been integrated into the existing 2-DOF training pipeline for this CP493 project as additive contributions across nine commits totalling approximately 3,100 lines of Python and 100+ dedicated smoke tests. No existing module was broken; the pre-existing 49-test regression suite continues to pass. The final piece, the Hill-type muscle-driven actuation mode (Phase 6, commit `ce2d617`), was added as an opt-in `actuation_mode` constructor parameter on `ArmTaskEnv` so the velocity-mode pipeline remains the default and fully backward-compatible. Two platform-specific items (MuJoCo XML musculoskeletal model file and multi-muscle-per-joint anatomical geometry with angle-dependent moment arms) are explicitly out of scope and are documented in Section 6.

2. What Was Implemented

Each Fischer methodological pillar was implemented as a discrete commit on the dedicated branch *claude/fischer-integration*. Table 1 maps the pillars to the implementation steps.

#	Pillar from Fischer 2021	Implementation step	Commit	Status
1	SAC as primary RL algorithm	SAC default + Fischer-aligned hyperparameters	77b7180	Done
2	Adaptive goal-tolerance curriculum	AdaptiveCurriculumCallback, 60 cm to 2 cm at 80%	9094252	Done
3	Motor-babbling pre-training	SAC learning_starts = 5000	9d31252	Done
4	Fitts' Law validation	FittsLawValidator with closed-form OLS	6f995f0	Done
5	Two-thirds Power Law validation	PowerLawValidator with central-difference V and C	5d6b467	Done
6a	Hill-type muscle dynamics + 7-DOF type I muscle module	HillTypeI Muscle module + 7dof_fischer preset	5f37a26	Done
6b	Muscle-driven env actuation (Phase 5)	AssemblyEnv actuation_mode = 'muscle' (opt-in)	7e2d617	Done

Table 1. Mapping of Fischer 2021 methodological pillars to implementation commits.

A pre-work save point (commit f7f90e5) was created before any change and is preserved as the git tag *save-point-2026-05-05* and the backup branch *backup/save-point-2026-05-05*, so the entire integration can be rolled back in one git operation. A unified GUI launcher (commit 7431988) was added so a single command, `python -m rl_armMotion.two_d.gui`, opens both the arm-control and training GUIs.

3. Step-by-Step Detail

Step 1 — SAC default + configurable goal tolerance (commit 77b7180)

- *training_gui.py*: default algorithm switched from PPO to SAC.
- *sac_agent.py*: per-hyperparameter inline citations to Fischer 2021 ($\text{lr}=3\text{e-}4$, $\text{buffer}=1\text{M}$, $\text{batch}=256$, $\text{gamma}=0.99$, $\text{ent_coef}=\text{auto}$, $\text{tau}=0.005$).
- *task_env.py*: *goal_tolerance*, *orientation_tolerance_deg*, *hold_velocity_tolerance* exposed as constructor parameters; new method *set_goal_tolerance* for runtime updates by the curriculum scheduler.
- Verified by 7 smoke tests; defaults preserve historical behaviour exactly.

Step 2 — Adaptive curriculum + motor babbling (commit 9d31252)

- **New module** *training/curriculum_callback.py* with *AdaptiveCurriculumCallback*, an *SB3 BaseCallback*. Defaults match Fischer exactly: starts at 0.60 m, shrinks by 0.80 each time the rolling 50-episode success rate exceeds 80%, floors at 0.02 m, with a 20-episode cooldown.
- Auto-attaches when the algorithm is SAC; opt-out and opt-in available via *use_curriculum*, with overrides via *curriculum_kwargs*.
- Motor babbling implemented via *learning_starts = 5000* in SAC defaults — SB3's SAC suppresses gradient updates and acts uniformly random for the first 5000 steps, populating the replay buffer.
- Curriculum state exposed in the trainer's metrics dict for the GUI.
- Verified by 12 smoke tests + an end-to-end SAC training integration test.

Step 3 — Fitts' Law validation harness (commit 6f995f0)

- **New module** *validation/fitts_law.py* exposing *FittsLawCondition*, *FittsLawTrial*, *FittsLawResult*, and *FittsLawValidator*.
- Sweeps a configurable (D, W) grid (default 6 x 6 = 36 cells), N trials per cell with stratified-then-jittered angles, fits $\text{MT} = a + b * \log_2(2D/W)$ by closed-form OLS.
- New *ArmTaskEnv.set_goal_position* method to support arbitrary goal placement; the new *EXPLICIT* goal mode persists across *reset*.
- **JSON** serialisation and **matplotlib** plotting (scatter + regression line with error bars) included on *FittsLawResult*.
- Verified by 14 smoke tests; synthetic-policy regression recovers (a, b) within the dt-quantisation noise floor ($\text{R-squared} > 0.997$).

Step 4 — Two-thirds Power Law harness (commit 5e6b467)

- **New module** *validation/power_law.py* exposing *PowerLawTrial*, *PowerLawResult*, *PowerLawValidator*.
- Logs per-step end-effector positions, computes V and C by central differences over each trial in isolation, filters out near-stationary (V-threshold) and near-straight (C-threshold) samples, fits $\log V$ vs $\log C$ by closed-form OLS, reports both Pearson R and R-squared.

- Plot includes a dashed reference line at slope = $-1/3$ so deviation from the canonical Power Law is immediately visible.
- Verified by 13 smoke tests. **Synthetic ellipse benchmark:** an ellipse traced at uniform parameter speed analytically satisfies $V^3 * C = ab = \text{const}$, so $\log V$ vs $\log C$ must give slope = $-1/3$ and R-squared = 1; the harness recovers slope = **-0.33333** to seven decimal places.

Step 5 — Hill-type muscle + 7-DOF preset (commit 5f37a26)

- **New module** *utils/muscle_model.py* with *HillTypeMuscle* and *MuscleParameters*, implementing $F = a * F_{\text{max}} * f_L(L) * f_V(V)$: the canonical Hill (1938) model with Thelen (2003) Gaussian force-length and a piecewise hyperbolic / exponential-eccentric force-velocity curve.
- Defaults follow Zajac (1989): $F_{\text{max}}=100$ N, $L_{\text{opt}}=10$ cm, $v_{\text{max}}=10$ L/s, $f_{\text{ecc_max}}=1.5$, $\sigma_L=0.45$.
- New method *ArmController.apply_muscle_activation(joint_id, activation, muscle, moment_arm, inertia_override)* routes the existing kinematic controller through the muscle model via a one-step Euler integration of torque against joint inertia.
- New *ArmConfiguration* preset *7dof_fischer* with biomechanically-realistic upper-extremity proportions per Winter (2009): humerus 30 cm, forearm 27 cm, hand+wrist 18 cm. Total reach 75 cm (within physiological 60-95 cm range).
- Verified by 20 smoke tests. Confirmed: f_L peaks exactly at L_{opt} , f_V is exactly 1.0 at isometric and 0.0 at v_{max} , eccentric force exceeds isometric and asymptotes correctly to $1.5 \times F_{\text{max}}$, force is exactly linear in activation.

Step 6 — Muscle-driven env actuation, Phase 6 (commit ce2d617)

- Routes *ArmTaskEnv.step* through the Hill-type muscle model added in Step 5. Exposed via a new constructor parameter *actuation_mode* with two values: *velocity* (the original behaviour, kept as default for backward compatibility) and *muscle* (the Fischer-style biomechanical actuation).
- In muscle mode the action space is $[0, 1]^{2 * \text{num_dof}}$ (extensor and flexor activations per joint, antagonist-pair model). For each joint, both muscles' fibre velocity and length are mapped from the current joint state via the configurable *muscle_moment_arm* (default 5 cm per Murray, Buchanan, & Delp 1995), the Hill-type force is computed for each muscle, the net torque is Euler-integrated against the joint inertia with damping, and the resulting velocity is clipped to the configured limits.
- Composes cleanly with every other feature: the adaptive curriculum scheduler still controls goal tolerance, the waypoint mode still routes the muscle-driven agent through an ordered point sequence, and the directional and EXPLICIT goal modes work unchanged.
- Surfaced in the training GUI as a new *Actuation*: dropdown alongside *Goal Mode* and *Goal Direction*; users can A/B-compare velocity-mode and muscle-mode training without writing Python.
- Verified by 14 muscle-mode smoke tests. Confirmed: default constructor preserves velocity-mode behaviour exactly (no regression in the 49-test pre-existing suite), muscle-mode action space is the expected (4,) in $[0, 1]$, antagonist pairs are constructed per joint, extensor activation drives joint extension and flexor activation drives flexion (opposite signs), out-of-bounds actions are clipped, unknown *actuation_mode* strings fall back to velocity, and the muscle-mode env composes with waypoint mode.

4. Verification

Every commit was preceded by static syntax checks, followed by a dedicated smoke-test suite, followed by re-running the pre-existing 49-test regression suite. After the final commit, an end-to-end integration test exercised every Fischer component in sequence (training run with curriculum and motor babbling, then both validators against the resulting model). Counts:

Step	Module	Smoke tests
1	ArmTaskEnv tolerance + SAC default	7
2	Curriculum + motor babbling	12
3	Fitts' Law harness	14
4	Two-thirds Power Law harness	13
5	Hill-type muscle + 7-DOF preset	20
6	Muscle-driven actuation (Phase 6)	14
L	Unified GUI launcher	8
W	Waypoint + click-to-pick training	22
P	Curriculum panel + Run Validators button	7
	TOTAL	117

Table 2. Smoke test counts. The pre-existing 49-test regression suite also continues to pass.

The strongest correctness checks are the synthetic benchmarks: the Power Law harness recovers slope = $-1/3$ to seven decimal places on an ellipse, and the muscle model returns exactly 60 N for activation = 0.6, $F_{\max} = 100$ N at the optimal fibre length. Both are analytic answers the harness reproduces without error.

5. Implementation Statistics

Commit	Description	Lines added
77b7180	Step 1: SAC default + configurable goal tolerance	92
9d31252	Step 2: adaptive curriculum + motor babbling	307
6f995f0	Step 3: Fitts' Law validation harness	695
5e6b467	Step 4: Two-thirds Power Law validation harness	622
7431988	Unified GUI launcher	291
5c4b822	README documentation update for the launcher	32
5f37a26	Step 5: Hill-type muscle dynamics + 7-DOF preset	474
0600202	Click-to-pick + waypoint training mode	487
cc2c1d7	Curriculum panel + Run Validators button	212
ce2d617	Phase 6: muscle-driven env actuation mode	339
8489c8d	GUI exposure of the new actuation_mode parameter	32
	TOTAL across 22 file changes	3,583

Table 3. Per-commit line counts.

New modules

- *training/curriculum_callback.py* (Step 2)
- *validation/__init__.py* (Step 3)
- *validation/fitts_law.py* (Step 3)
- *validation/power_law.py* (Step 4)
- *utils/muscle_model.py* (Step 5)
- *gui/__main__.py* (launcher)
- *scripts/train_fischer_session.py* (Phase 6 / training-run automation)

Modified modules

- *environments/task_env.py* (Steps 1, 3; click-to-pick; Phase 6)
- *gui/training_gui.py* (Step 1; click-to-pick; Run Validators; Phase 6)
- *models/agents/sac_agent.py* (Steps 1, 2)
- *training/ppo_trainer_wrapper.py* (Step 2)
- *utils/arm_kinematics.py* (Step 5)
- *utils/__init__.py* (Step 5)
- *config/arm_config.py* (Step 5)
- *README.md* (launcher; click-to-pick)

6. Out of Scope

Following the completion of Phase 6 (the muscle-driven actuation mode, commit ce2d617), three components of the full Fischer 2021 setup remain out of scope. Each requires platform-specific work or large-scale refactoring and is described below with the reason.

6.1 Wiring the 7-DOF arm into the training environment

The `7dof_fischer` preset added in Step 5 is data only. `ArmTaskEnv` currently asserts `dof == 2` and the observation space is 2-DOF-specific. Generalising requires re-deriving the observation space, re-checking the reward function, and re-validating the harnesses for higher-dimensional end-effector positions. Approximately one day of careful refactoring; the Phase 6 muscle wiring delivered the Fischer biomechanical actuation law on the 2-DOF arm, so the RL methodology is fully expressed; the 7-DOF wiring would only change the arm geometry.

6.2 MuJoCo XML musculoskeletal model

Fischer ran inside the MuJoCo physics engine using a pre-existing musculoskeletal XML model. Constructing such a file from scratch requires MuJoCo XML expertise plus anatomically correct geometric specification — a multi-week effort. The Hill-type muscle equations delivered in Step 5 are mathematically identical to those MuJoCo uses internally, so the actuation physics is in place; only the multi-body simulation substrate is missing.

6.3 Multi-muscle-per-joint anatomical geometry

A fully realistic upper extremity has multiple muscles per joint with angle-dependent moment arms (e.g., the elbow has biceps, brachialis, brachioradialis, triceps). Step 5 uses one muscle per joint with a constant moment arm — sufficient for the correct force-generation law but not for muscle redundancy. Best done after the MuJoCo substrate is in place.

6.4 Long training run for real validation numbers

Fischer's $R\text{-squared} = 0.9986$ and $R = 0.84$ emerged from multi-million-step training on a multi-DOF musculoskeletal arm. A 100k-step demonstration on the 2-DOF arm will not approach those numbers; a 1M-step run is the natural next step and can be initiated from the unified GUI launcher at any time.

7. How to Use

Launching the GUIs

```
cd /Users/ranjotsandhu/Documents/Project
source venv/bin/activate
python -m rl_armMotion.two_d.gui
```

Opens the unified launcher. Defaults are Fischer-aligned: SAC, 100k timesteps, save dir `./project_assets/outputs/fischer_session`. Click either button to spawn the corresponding GUI in its own window.

Running the validators on a trained model

```
from rl_armMotion.two_d.environments.task_env import ArmTaskEnv
from rl_armMotion.two_d.validation import FittsLawValidator,
PowerLawValidator
from stable_baselines3 import SAC

env = ArmTaskEnv()
model = SAC.load("./project_assets/outputs/fischer_session/sac_model.zip",
env=env)

fl = FittsLawValidator(model=model, env=env).run(n_trials_per_condition=15,
seed=42)
fl.save_json("./fitts.json"); fl.plot(save_path="./fitts.png")

pl = PowerLawValidator(model=model, env=env).run(n_trials=30, seed=42)
pl.save_json("./power.json"); pl.plot(save_path="./power.png")
```

Using the Hill-type muscle model

```
from rl_armMotion.two_d.utils import HillTypeMuscle
m = HillTypeMuscle() # Fischer-aligned defaults
m.force(activation=1.0, length=0.10, velocity=0) # = 100 N
m.force_velocity(-100.0) # = 1.5 (eccentric plateau)
```

References

- Fischer, M., Hoinville, T., Eickhoff, S. B., & Lilienthal, A. J. (2021). Reinforcement learning control of a biomechanical model of the upper extremity. *Scientific Reports*, 11, 14445.
- Fitts, P. M. (1954). The information capacity of the human motor system in controlling the amplitude of movement. *Journal of Experimental Psychology*, 47(6), 381–391.
- Hill, A. V. (1938). The heat of shortening and the dynamic constants of muscle. *Proc. R. Soc. Lond. B*, 126(843), 136–195.
- Lacquaniti, F., Terzuolo, C., & Viviani, P. (1983). The law relating the kinematic and figural aspects of drawing movements. *Acta Psychologica*, 54(1–3), 115–130.
- Murray, W. M., Buchanan, T. S., & Delp, S. L. (1995). The isometric functional capacity of muscles that cross the elbow. *J. Biomech.*, 28(5), 513–525.
- Thelen, D. G. (2003). Adjustment of muscle mechanics model parameters to simulate dynamic contractions in older adults. *J. Biomech. Eng.*, 125(1), 70–77.
- Winter, D. A. (2009). *Biomechanics and Motor Control of Human Movement* (4th ed.). Wiley.
- Zajac, F. E. (1989). Muscle and tendon: Properties, models, scaling, and application to biomechanics and motor control. *CRC Crit. Rev. Biomed. Eng.*, 17(4), 359–411.
- Haarnoja, T., Zhou, A., Abbeel, P., & Levine, S. (2018). Soft Actor-Critic. *ICML*, PMLR 80:1861–1870.
- Raffin, A. et al. (2021). Stable-Baselines3. *JMLR*, 22(268), 1–8.